Relatório - Sistema de transações

Membros do grupo:

Lucas dos Santos
Aron Garcia
Pedro Senne
Jean Wander

- **Introdução:** Desenvolver um sistema, em Java, de análise de transações de formas: Sequencial, Concorrente e Concorrente usando afinidade. O objetivo do sistema é servir como experimentação prática das formas diferentes de otimização e programação de tal sistema.

- **Estrutura do conjunto de dados:** A estrutura do conjunto se encontra em uma lista de objetos da classe Transação, que possui atributos armazenados como dados primitivos. Tais dados são gerados por uma classe geradora que produz dados de forma dinâmica e determinística.

- **Algoritmos:** O algoritmo sequencial percorre uma lista de objetos Transação gerados pelo gerador, sem a divisão por threads

```java
package com.example;

import java.util.List;

public class Sequencial {
    public static void main(String[] args) {
        List<Transacao> dados = geraDados.gerar(1_000_000, 42);

        for (int i = 0; i < 3; i++) Estatistica.de(dados);

        long inicio = System.nanoTime();
        Estatistica e = Estatistica.de(dados);
        long ms = (System.nanoTime() - inicio) / 1_000_000;

        System.out.println("====================================");
        System.out.println("         VERSAO SEQUENCIAL");
        System.out.println("====================================");
        System.out.println("Soma dos valores: " + e.soma);
        System.out.println("Média dos valores: " + e.media());
        System.out.println("Maior transação: " + e.maior);
        System.out.println("Menor transação: " + e.menor);
        System.out.println("IDs das transações suspeitas: " + e.idsSuspeitos);
        System.out.println("Tempo (sequencial): " + ms + " ms");
        System.out.println("====================================");
    }
}
```

Já o ExecService usa da forma concorrente, que simula uma “paralelização”, dividindo o processamento das transações por threads, presentes em cores, do processador (CPU).

```java
package com.example;

import java.util.concurrent.Executors;
import java.util.concurrent.ExecutorService;
import java.util.List;


public class ExecService {
    public static void main(String[] args) throws Exception{
        int nThreads = 4;
        List<Transacao> dados = geraDados.gerar(1_000_000, 42);
        ExecutorService executor = Executors.newFixedThreadPool(nThreads);

        try {
            for (int i = 0; i < 3; i++) Paralelo.calcular(dados, executor, nThreads);

            long inicio = System.nanoTime();
            Estatistica e = Paralelo.calcular(dados, executor, nThreads);
            long ms = (System.nanoTime() - inicio) / 1_000_000;

            System.out.println("====================================");
            System.out.println("        VERSAO EXECUTORSERVICE");
            System.out.println("====================================");
            System.out.println("Soma dos valores: " + e.soma);
            System.out.println("Média dos valores: " + e.media());
            System.out.println("Maior transação: " + e.maior);
            System.out.println("Menor transação: " + e.menor);
            System.out.println("IDs das transações suspeitas: " + e.idsSuspeitos);
            System.out.println("Tempo (ExecutorService): " + ms + " ms");
            System.out.println("====================================");

        } finally {
            executor.shutdown();
        }
    }
}
```

Por último, o AffinityService implementa o sistema aplicando o conceito de afinidade, ou seja, reservando threads para tarefas específicas.

```java
package com.example;

import net.openhft.affinity.AffinityStrategies;
import net.openhft.affinity.AffinityThreadFactory;

import java.util.List;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class AffinityService {
    public static void main(String[] args) throws Exception {
        int nThreads = 4;
        List<Transacao> dados = geraDados.gerar(1_000_000, 42);
        ExecutorService executor = Executors.newFixedThreadPool(nThreads,
                new AffinityThreadFactory("worker", AffinityStrategies.DIFFERENT_CORE));

        try {
            for (int i = 0; i < 3; i++) Paralelo.calcular(dados, executor, nThreads); // warm-up

            long inicio = System.nanoTime();
            Estatistica e = Paralelo.calcular(dados, executor, nThreads);
            long ms = (System.nanoTime() - inicio) / 1_000_000;

            System.out.println("====================================");
            System.out.println("     VERSAO EXECUTOR + AFFINITY");
            System.out.println("====================================");
            System.out.println("Soma dos valores: " + e.soma);
            System.out.println("Média dos valores: " + e.media());
            System.out.println("Maior transação: " + e.maior);
            System.out.println("Menor transação: " + e.menor);
            System.out.println("IDs das transações suspeitas: " + e.idsSuspeitos);
            System.out.println("Tempo (ExecutorService + Affinity): " + ms + " ms");
            System.out.println("====================================");
        } finally {
            executor.shutdown();
        }
    }
}
```

- **Experimentos:** Os teste do sistemas foram feitos através da execução em um dataset de teste básico com 1 milhão de dados gerados de forma dinâmica, medindo os valores de transação, além de verificar as transações por características suspeitas, as quais foram definidas de maneira arbitrária. Além disso, foram criadas classes auxiliares para análises estatísticas às quais se calculam os valores de categorias específicas.

- **Resultados estatístico por categoria de transação:**
  A classe EstatisticaCategoria mantém os acumuladores de cada categoria. A quantidade é incrementada a cada registro; o valor é acumulado em centavos (long), evitando diferenças de arredondamento na soma; e a maior transação é atualizada quando um valor superior é encontrado. A média é obtida dividindo o valor total pela quantidade de transações.

Foram implementadas três estratégias de processamento:

Sequencial: percorre as transações uma a uma e atualiza o acumulador da categoria correspondente.

ExecutorService: divide o conjunto em partes, executadas por tarefas em threads de um pool. Cada tarefa produz estatísticas parciais, que são reunidas ao final.

ExecutorService com Affinity: mantém a estratégia de tarefas paralelas e utiliza a biblioteca OpenHFT Affinity na criação das threads.

Na versão paralela, Callable permite que cada tarefa retorne seu mapa parcial; Future permite recuperar esse resultado; e o método combinar() reúne quantidade, total e maior valor por categoria.

Resultado demonstrativo – 10 milhões de transações

Categoria	Quantidade	Valor total	Valor médio	Maior transação
Alimentação	2.001.005	R$ 100.034.344.023,30	R$ 49.992,05	R$ 100.000,00
Educação	2.000.574	R$ 100.030.018.386,64	R$ 50.000,66	R$ 99.999,98
Eletrônicos	1.999.260	R$ 100.061.176.321,71	R$ 50.049,11	R$ 99.999,92
Saúde	1.999.118	R$ 99.907.079.374,80	R$ 49.975,58	R$ 99.999,99
Transporte	2.000.043	R$ 100.001.389.058,30	R$ 49.999,62	R$ 99.999,95

Na execução registrada, foram processadas 10.000.000 transações sintéticas. A conferência automática informou que as estatísticas das versões ExecutorService e ExecutorService com Affinity são iguais às da versão sequencial. A soma das quantidades por categoria também correspondeu ao total de registros gerados.

O teste demonstra que o módulo calcula as quatro estatísticas solicitadas por categoria e que, na execução registrada, as três estratégias produziram resultados equivalentes segundo o validador. O maior volume concluído e conferido nesta execução foi de 10 milhões de transações

- **Speedup e Eficiência Paralela:**

Métricas 2 Threads			
Medidas	1.000.000 transações	9.500.000 transações	19.000.000 transações
Tempo Sequencial	36 ms	112 ms	201 ms
ExecutorService	28 ms	97 ms	191 ms
Tempo com Affinity	185 ms	100 ms	161 ms
Speed Up	Executor: 1,13X Affinity: 0,19X	Executor: 1,12X Affinity: 1,12X	Executor: 1,052X Affinity: 1,23X
Eficiência Paralela	Executor: 53,6% Affinity: 9,7%	Executor: 56% Affinity: 54,4%	Executor: 48,5% Affinity: 55%

Métricas 4 Threads			
Medidas	1.000.000 transações	9.500.000 transações	19.000.000 transações
Tempo Sequencial	36 ms	112 ms	201 ms
ExecutorService	20 ms	72 ms	143 ms
Tempo com Affinity	15 ms	73 ms	146 ms
Speed Up	Executor: 1,8X Affinity: 2,4X	Executor: 1,55X Affinity: 1, 53X	Executor: 1,41X Affinity: 1,37X
Eficiência Paralela	Executor: 45% Affinity: 60%	Executor: 37,5% Affinity: 36,1%	Executor: 35,1% Affinity: 34,2%

Métricas 8 Threads			
Medidas	1.000.000 transações	9.500.000 transações	19.000.000 transações
Tempo Sequencial	36 ms	112 ms	201 ms
ExecutorService	7 ms	52 ms	101 ms
Tempo com Affinity	7 ms	53 ms	112 ms
Speed Up	Executor: 5,14X Affinity: 5,14X	Executor: 2,12X Affinity: 2,15X	Executor: 1,79X Affinity: 1,99X
Eficiência Paralela	Executor: 64,25% Affinity: 64,25%	Executor: 25,6% Affinity: 25,9%	Executor: 20,9% Affinity: 21,5%

- **Conclusão e Analise Dos Resultados**

Com base nos dados obtidos, percebe-se que ambos a execução do sistema através de ExecutorService assim como a implementação de Afinidade com núcleos tendem a aumentar significativamente a velocidade e eficiência do sistema. Percebe-se também que aumento de velocidade resultando da implementação de afinidade não é linear, com um efeito reduzindo após certo ponto, tal redução da efetividade da adição de afinidade é notável no teste executado com 8 threads, no qual a adição de affinity teve efeitos mínimos na velocidade, só tendo resultados visíveis em valores mais altos de transações.